Plugins for AI Agents: One Bundle, Your Whole Team

Mode 2 Phase 1 (Building Blocks), Chapter 5/6 — Revision & Viva-Prep Cheatsheet (English + Roman Urdu)
Source: agentfactory.panaversity.org/docs/plugins-crash-course

EN: A plugin bundles skills, subagents, MCP servers, instructions, and hooks so a generic coding agent (Claude Code/OpenCode) becomes YOUR TEAM's own, with one install. The test for needing a plugin isn't what it does -- it's who else needs it.
Roman Urdu: Plugin skills, subagents, MCP servers, instructions, hooks ka bundle hai jo generic coding agent (Claude Code/OpenCode) ko ek install se AAPKI TEAM ka bana deta. Plugin chahiye ka test 'yeh kya karta' nahi -- 'kaun aur isay chahiye' hai.

1. The Shape + 2 Hosts

- A plugin is pieces the HOST loads and runs -- the host owns the agent loop, model, machine; the plugin only contributes capabilities/rules.
- Claude Code = declarative bundle (folder + manifest); OpenCode = code module hooking into events. Most pieces port between them -- hooks are the exception (no shared format).

2. Three Capability Levers (all advisory)

- **Skills:** model-invoked by matching the description field -- the most important line, saying WHEN to use it, not just what it is.
- **Subagents:** helpers with their own clean context window -- use for self-contained, verifiable tasks; delegation has a cost.
- **MCP servers:** the plugin ships a POINTER to a remote server (logic/data/secrets stay on your infra) -- a local server is almost never the right choice.

3. The Deterministic Lever: Hooks

- A hook runs automatically at a fixed lifecycle point -- not a model suggestion. **Exit code 2 on PreToolUse = block** (the whole guardrail game); exit 1 does NOT block (the most common mistake).
- 'Must-always' needs a hook, not an instruction -- anything written in a skill/CLAUDE.md is advisory and can be forgotten.

4. Shipping + Trust

- Manifest: .claude-plugin/plugin.json holds ONLY the manifest -- everything else (skills/, agents/, hooks/) sits at the plugin root.
- A marketplace is just a git repo with a catalog file -- no package registry. You CAN charge for hosted access, not for plaintext files (no DRM on skills).
- Installing a plugin means running someone else's code -- review via claude plugin details before enabling anything.

Key Terms Cheatsheet

One line each — English + Roman Urdu.

Term	English	Roman Urdu
Plugin	A bundle (skills+subagents+MCP+instructions+hooks) the host loads to extend itself.	Bundle (skills+subagents+MCP+instructions+hooks) jo host load kar ke khud ko extend karta.
Advisory vs Deterministic	Skills/subagents/MCP are advisory (model decides); hooks are deterministic (always run).	Skills/subagents/MCP advisory hain (model decide karta); hooks deterministic hain (hamesha chalte).
Exit code 2 (PreToolUse)	Blocks the tool call -- the entire guardrail mechanism.	Tool call block karta -- poora guardrail mechanism.
Marketplace	A plain git repo with a catalog file listing plugins -- not a package registry.	Plain git repo jisme catalog file plugins list karti -- package registry nahi.
Test for needing a plugin	Not what it does, but who else needs it (travel beyond one repo).	Yeh kya karta nahi, balke kaun aur isay chahiye (ek repo se bahar travel).

Quick Self-Check

Question	Answer
What's the test for whether something belongs in a plugin vs a local .claude/ folder?	Whether the customization needs to TRAVEL (to teammates, other projects, the community) -- not what it technically does.
Why is the SKILL.md description field called 'the most important line'?	It's what decides when the skill fires -- it must say WHEN to use it, not just describe what it is.
What's the most common hook mistake, and the fix?	Using exit code 1 expecting it to block -- only exit code 2 on PreToolUse actually blocks the tool call.
Why should 'must-always' rules be hooks, not instructions in CLAUDE.md?	Anything in a skill or rules file is advisory and can be forgotten by the model -- a hook always runs.
What's the correct way to monetize a plugin, given skills are plaintext with no DRM?	Sell hosted access to a server the plugin points to, not the files themselves -- the plugin ships as a thin free client gated by a subscription key.

Where This Leads / Sources

Leads to: Build AI Agents (the loop isn't yours), AI Identity (identity isn't yours), connector-native apps (mirror image). Full worked example (agent-factory plugin) in Part 6.

Source: docs/plugins-crash-course/ (this repo), content originally via Zia Tutor AI connector.